

Manual técnico — Agente CFDI

Qué es este documento: cómo funciona *hoy* el sistema, qué servicios de GCP toca y cómo se relacionan. No es el diseño deseado ni el plan: es el estado verificado.

Verificado el 17-ago-2026 contra el proyecto real `project-d0428141-1b39-47af-9bc` (número `1031368580327`) con el SDK portátil de esta máquina. Cada afirmación de infraestructura de este manual sale de un comando reproducible; los comandos están en el §11.

1. La primera cosa que hay que entender: son dos aplicaciones

El repositorio contiene **dos aplicaciones FastAPI independientes**, y **solo una está desplegada**. Confundirlas es el error más caro al leer este proyecto.

Aspecto	Servicio del agente	Servicio de auditoría
Punto de entrada	<code>main.py</code>	<code>src/agente_cfdi/api/app.py</code>
Qué hace	Conversa vía Gemini con ADK	CFDI, bitácora, Merkle, cesiones
Dependencias	<code>google-adk</code> , <code>google-genai</code>	<code>httpx</code> , <code>sqlite3</code> , <code>stdlib</code>
Estado	[OK] desplegado en Cloud Run	[] solo corre en local y en CI
Lógica de negocio	ninguna	toda
Líneas del núcleo	~130	~2 500

Es decir: **la URL pública no expone la auditoría**. Lo que responde `https://agente-cfdi-run-xsxcmt7edq-uc.a.run.app` es un chatbot con la instrucción de sistema del agente y `tools=[]` — cero herramientas conectadas (`agente/agent.py:45`). El motor verificable —hashes, cadena, Merkle, doble cesión— existe, tiene pruebas y **no está en producción**.

Unificarlas es la tarea 1.13 en el plan del equipo y es prerequisite de todo lo demás.

2. Mapa de servicios de GCP

2.1 Lo que está encendido y en uso

Servicio	Recurso concreto	Papel
Cloud Run	agente-cfdi-run · us-central1 · rev. agente-cfdi-run-00001-2x8	Único cómputo. Sirve el servicio del agente
Vertex AI	aiplatform.googleapis.com · ubicación global	Inferencia de gemini-3.5-flash
Cloud Scheduler	job-cierre-diario · us-central1	Dispara el cierre diario a las 23:59 CDMX
Cloud Build	build 90b315dd-...	Compila el Dockerfile desde fuente
Artifact Registry	cloud-run-source-deploy · us-central1 · 72 MB	Guarda la imagen resultante
Cloud Storage	run-sources-project-...-us-central1	Recibe el zip del código en cada --source .
Cloud Logging	—	Logs de request de Run y de intento de Scheduler
IAM	SA 1031368580327-compute@developer.gserviceaccount.com	Identidad de build y de ejecución

2.2 Lo que está encendido y NO en uso

Servicio	Situación
Secret Manager	El secreto WALLET_PRIVATE_KEY existe (creado 17-ago 00:54) pero tiene 0 versiones : está vacío. Cloud Run no lo monta. No hay wallet, porque no hay anclaje real
Gemini API (generativelanguage)	Vía abandonada. El prepago de AI Studio se agotó con 429 RESOURCE_EXHAUSTED; se migró a Vertex
Cloud Monitoring	API encendida, cero políticas de alerta . Si el job falla mañana, nadie se entera
Cloud SQL	sqladmin.googleapis.com ni siquiera está habilitado. No hay base de datos administrada

El resto de las ~60 APIs habilitadas (BigQuery, Pub/Sub, Dataplex, IAP...) vienen encendidas por plantilla del proyecto y **el código no las toca**.

2.3 Cómo se relacionan

DIAGRAMA (fuente Mermaid — ver la version interactiva del manual para el render visual)

```
graph TB
    subgraph dev["Máquina de desarrollo"]
        PS["deploy.ps1<br/>gcloud run deploy --source ."]
    end

    subgraph gcp["GCP · project-d0428141-1b39-47af-9bc"]
        GCS["Cloud Storage<br/>run-sources-...-us-central1<br/>zip del código"]
        CB["Cloud Build<br/>compila el Dockerfile"]
        AR["Artifact Registry<br/>cloud-run-source-deploy"]
        RUN["Cloud Run · us-central1<br/>agente-cfdi-run<br/>1 vCPU · 512 Mi · maxScale 20"]
        VX["Vertex AI · global<br/>gemini-3.5-flash"]
        SCH["Cloud Scheduler · us-central1<br/>job-cierre-diario"]
        LOG["Cloud Logging"]
        SM["Secret Manager<br/>WALLET_PRIVATE_KEY<br/>0 versiones · sin uso"]
        SA["Service Account<br/>1031368580327-compute"]
    end

    INT["Internet<br/>allUsers = run.invoker"]
    CF["CØRD Fiscal<br/>plataforma preexistente<br/>HTTPS + JWT"]

    PS -->|"sube fuente"| GCS
    GCS --> CB
    CB -->|"push imagen"| AR
    AR -->|"despliega revisión"| RUN
    SCH -->|"POST /api/cierre-diario<br/>23:59 CDMX · sin OIDC"| RUN
    INT -->|"GET / · POST /api/chat"| RUN
    RUN -->|"ADC · generate_content"| VX
    RUN -.->|"stdout/stderr"| LOG
    SCH -.->|"AttemptStarted/Finished"| LOG
    SA -.->|"identidad de build y de runtime"| CB
    SA -.-> RUN
    SM -.->|"no montado"| RUN
    RUN -.->|"NO desplegado hoy:<br/>el servicio de auditoría<br/>consumiría los libros"| CF

    classDef muerto stroke-dasharray:4 4,opacity:0.55
    class SM,CF muerto
```

Lo que este diagrama dice y conviene no perder de vista:

- No hay base de datos en la nube. Nada.
- Cloud Run no lee ningún secreto. Su única credencial es la ADC de la cuenta de servicio, y solo la usa contra Vertex.
- La flecha hacia CØRD Fiscal es punteada porque **el servicio que la usaría no está desplegado**.
- Scheduler y el tráfico público entran por la misma puerta abierta.

2.4 Regiones: tres cosas distintas que se llaman parecido

Confundirlas rompe el arranque, y ya pasó una vez:

Cosa	Valor	Por qué
Región de Cloud Run	<code>us-central1</code>	Dónde corre el contenedor
Región de Scheduler	<code>us-central1</code>	Debe coexistir con el servicio
Ubicación del modelo	<code>global</code>	<code>gemini-3.5-flash</code> no está publicado en <code>us-central1</code> : ahí devuelve 404

`main.py:46-49` traduce `GOOGLE_CLOUD_REGION` (lo que documenta el README) a `GOOGLE_CLOUD_LOCATION` (lo que lee `google-genai`), con `global` por omisión.

3. Los tres recorridos que existen hoy

3.1 Despliegue

DIAGRAMA (fuente Mermaid — ver la version interactiva del manual para el render visual)

```
sequenceDiagram
    participant D as deploy.ps1
    participant G as Cloud Storage
    participant B as Cloud Build
    participant A as Artifact Registry
    participant R as Cloud Run

    D->>D: fija PATH al SDK portátil y CLOUDSDK_CONFIG
    D->>G: gcloud run deploy --source .
    Note over G: sube el zip filtrado por .dockerignore
    G->>B: dispara build
    B->>B: docker build sobre python:3.11-slim
    B->>A: push de la imagen
    A->>R: crea revisión
    R->>R: startup probe TCP :8080
    R-->>D: URL pública
```

El `.dockerignore` deja fuera `.env`, `.venv`, `docs/`, `smoke_test*.py`, `deploy.ps1` y el README. **La imagen no lleva secretos horneados** — esto es correcto y hay que mantenerlo así.

Detalle operativo: el script fija `CLOUDSDK_CONFIG=D:\CORD\tools\gcloud-config-ricardo` porque en esta máquina hay dos cuentas autenticadas. Desplegar sin eso puede apuntar al proyecto equivocado.

Permiso que hubo que dar una vez — en proyectos nuevos la SA de Compute no lo recibe sola y el deploy falla con 403 sobre `run-sources-*`:

```
gcloud projects add-iam-policy-binding project-d0428141-1b39-47af-9bc
--member="serviceAccount:1031368580327-compute@developer.gserviceaccount.com"
```

```
--role="roles/cloudbuild.builds.builder"
```

3.2 Una petición de chat

DIAGRAMA (fuente Mermaid — ver la version interactiva del manual para el render visual)

```
sequenceDiagram
    participant U as Cualquiera en internet
    participant R as Cloud Run · main.py
    participant S as InMemorySessionService
    participant AG as root_agent · LlmAgent
    participant V as Vertex AI · global

    U->>R: POST /api/chat {"message": "..."}
    R->>R: session_id recibido o uuid4 nuevo
    R->>S: recupera o crea la sesión
    R->>AG: Runner.run_async
    AG->>V: generate_content · gemini-3.5-flash
    V-->>AG: respuesta
    AG-->>R: evento final
    R-->>U: {reply, session_id, model}
```

Tres cosas que hay que saber de este recorrido:

1. **La sesión vive en memoria.** Al reciclar la instancia se pierde el hilo de conversación. Es aceptable porque el estado que importa vive en la bitácora — pero la bitácora no está desplegada, así que hoy **no se persiste nada**.
2. **El agente no tiene herramientas.** `tools=[]`. No puede consultar la cadena ni verificar un folio; solo redacta.
3. Cualquier excepción se convierte en `500` con el texto de la excepción en el cuerpo (`main.py:104`). En un servicio público eso filtra detalle interno.

3.3 El cierre diario

DIAGRAMA (fuente Mermaid — ver la version interactiva del manual para el render visual)

```
sequenceDiagram
    participant S as Cloud Scheduler
    participant R as Cloud Run
    participant L as Cloud Logging

    Note over S: 23:59 America/Mexico_City = 05:59 UTC
    S->>R: POST /api/cierre-diario · sin OIDC
    R-->>S: 200 {"message": "Cierre diario ejecutado (simulado)"}
    S->>L: AttemptStarted + AttemptFinished 200
    R->>L: httpRequest 200 · 6.5 s de arranque en frío
```

Corrió por primera vez el 17-ago-2026 a las 05:59:00 UTC, sin consumir ninguno de sus 3 reintentos. La evidencia completa está en [docs/evidencias/2026-08-17-job-diario.md](https://docs.evidencias/2026-08-17-job-diario.md).

Lo que esto prueba es la plomería, no el cierre. `main.py:117` devuelve un literal con un `TODO` encima: no calcula Merkle, no ancla, no toca la bitácora.

4. Anatomía del repositorio

```
agente-cfdi-ricardo/
|-- main.py                <- DESPLEGADO. Transporte HTTP, sin lógica
|-- agente/agent.py        <- root_agent (LlmAgent de ADK), tools=[]
|-- Dockerfile             <- python:3.11-slim, uvicorn main:app :8080
|-- requirements.txt        <- lo que instala la IMAGEN (5 paquetes)
|-- pyproject.toml          <- lo que instala el CI (otro conjunto)
|-- deploy.ps1             <- gcloud run deploy --source .
|-- setup_sprint2.ps1      <- crea el secreto y el job de Scheduler
|-- smoke_test.py          <- verifica que el modelo responde
|
|-- src/agente_cfdi/        <- NO DESPLEGADO. El núcleo verificable
|   |-- api/app.py          <- 7 endpoints de auditoría
|   |-- api/dependencias.py <- conexión SQLite por petición, ancla, fuente
|   |-- dominio/canonico.py <- CORD-CANON-2, serialización congelada
|   |-- cfdi/lector.py      <- lector de CFDI 4.0 y rechazos tipificados
|   |-- bitacora/cadena.py  <- SHA-256 encadenado + árbol de Merkle
|   |-- bitacora/almacen.py <- persistencia append-only, anti-doble-cesión
|   |-- bitacora/anclaje.py <- protocolo Ancla + AnclaSimulada
|   |-- auditoria/cotejo.py <- CFDI contra los libros
|   |-- fuentes/           <- protocolo + sintética + cliente CORD Fiscal
|   |-- sintetico/         <- generador de CFDI de prueba
|
|-- migraciones/001_bitacora.sql <- esquema PostgreSQL, sin ejecutar en ningún lado
|-- tools/verificar_prueba.py   <- verificador independiente (solo stdlib)
|-- tools/demo.py              <- escenario completo, lo corre el CI
`-- docs/adr/                  <- 5 ADR con las decisiones y su porqué
```

Aviso sobre dependencias: `requirements.txt` (lo que corre en Cloud Run) y `pyproject.toml` (lo que corre en el CI) declaran conjuntos **distintos y disjuntos**. La imagen no lleva `httpx` ni `python-multipart` ni `pydantic` explícito; el CI no lleva `google-adk`. Hoy no truena porque cada aplicación usa solo su lado, pero al unificar los servicios (tarea 1.13) esto revienta al importar, no al llamar el endpoint. Es un pendiente concreto, no una opinión de estilo.

5. El núcleo de auditoría — cómo funciona, aunque no esté desplegado

Este es el trabajo real del proyecto y conviene entenderlo, porque el despliegue eventualmente lo tiene que servir.

5.1 El ciclo

DIAGRAMA (fuente Mermaid — ver la version interactiva del manual para el render visual)

```
graph LR
  A["1 · Ingesta<br/>lote de XML"] --> B["2 · Validación<br/>lector CFDI 4.0"]
  B --> C["3 · Auditoría<br/>cotejo contra libros"]
  C --> D["4 · Registro<br/>bitácora encadenada"]
  D --> E["5 · Detección<br/>¿ya fue cedido?"]
  E --> F["6 · Expediente<br/>dossier al financiador"]
  D --> G["7 · Merkle<br/>una raíz por día"]
  G --> H["8 · Anclaje<br/>SIMULADO hoy"]
  H --> I["9 · Prueba<br/>de inclusión"]

  classDef sim fill:#7a2e2e,color:#fff,stroke:#a94442
  class H sim
```

Los pasos 1–6 corren por lote; 7–9 los dispara el job diario.

5.2 Las cuatro propiedades que sostienen el sistema

a) Serialización canónica (CORD-CANON-2). Un mismo hecho produce siempre los mismos bytes. Sin esto, dos serializaciones distintas del mismo registro dan dos hashes distintos y la cadena deja de significar algo. Está congelada.

b) Encadenamiento. `hash_n = SHA256(0x00 || canónico_n || hash_{n-1})`. El génesis depende del inquilino: sin eso, los registros de una PYME podrían injertarse en la cadena de otra.

Los prefijos de byte (0x00 hoja, 0x01 nodo, 0x02 génesis) son la defensa estándar contra segunda preimagen en árboles de Merkle (RFC 6962). Cuestan un byte y agregarlos después invalidaría todo lo escrito.

c) Doble cesión. La garantía **no** es un `SELECT` antes del `INSERT` — eso tiene una carrera, y mandar dos solicitudes simultáneas es exactamente lo que hace quien quiere ceder dos veces. La garantía es la **clave primaria sobre el UUID a secas**, de alcance global y no por inquilino: un folio lo emite el SAT y pertenece a un solo emisor, así que acotarlo por inquilino lo derrotaría cualquiera que abra dos cuentas.

d) Separación entre lo que prueba y lo que identifica. Dos tablas:

Tabla	Contenido	Retención
<code>bitacora_cadena</code>	posición y hashes	eterna — no hay datos personales
<code>bitacora_registros</code>	el canónico, con RFC y montos	caduca

Suprimir contenido no rompe la cadena: los eslabones siguen enlazando y lo único que se pierde es poder recalcular ese hash. Por eso `/bitacora/verificacion` devuelve `recalculados` y `altura` por separado — «verifiqué 200» y «verifiqué 3 y confié en 197» no son lo mismo.

5.3 Endpoints (`src/agente_cfdi/api/app.py`)

Método	Ruta	Qué hace
POST	/ingesta	Lote de CFDI: lee, audita contra los libros, encadena
POST	/cesiones	Cede un folio a un financiador
GET	/cesiones/{uuid}	¿Está tomado? No dice a nombre de quién
GET	/bitacora/verificacion	Recorre la cadena y reporta integridad
POST	/bitacora/anclaje	Publica la raíz del día. Idempotente por día
GET	/auditoria/prueba/{uuid}	Prueba de inclusión verificable sin nosotros
GET	/salud	Altura y punta de la cadena

Códigos de estado con significado deliberado:

Situación	Código	Razón
Folio ya cedido a otro	409	Conflicto real
Folio ya cedido al mismo	200	Reintento idempotente, no fraude
Libros inalcanzables	503	Falla de red, no «sin respaldo»
Registro caducado por retención	410	La prueba ya no se puede verificar
Día sin registros al anclar	409	No hay nada que publicar

El 503 es el más importante del cuadro: si una caída de CØRD Fiscal devolviera veredictos `sin_respaldo`, el financiador leería una falla de infraestructura como libros inconsistentes.

Un hallazgo **no bloquea la cesión**: se cede y se advierte. Bloquear sería tomar por el financiador una decisión comercial que es suya; lo inaceptable es que no se entere.

5.4 El anclaje es simulado, y se declara

`AnclaSimulada` no publica nada. Marca su red con `simulada:` y la constancia lleva `verificable_por_terceros = False`, que viaja hasta la respuesta HTTP; el verificador independiente sale con código 2 en vez de 0.

La decisión de diseño: un ancla falsa que pareciera real sería **peor que ninguna**, porque pasaría por buena en un video de demo y nadie notaría la diferencia hasta buscarla en un explorador de bloques.

Conectar una red real es sustituir una clase por otra que cumpla el mismo protocolo `Ancla` (`bitacora/anclaje.py:69`). Lo que falta no es código: es decidir red, financiar gas y custodiar una llave. De ahí el secreto `WALLET_PRIVATE_KEY` vacío en Secret Manager.

5.5 La frontera con CØRD Fiscal

`ClienteCordFiscal` consume dos endpoints por HTTPS con un JWT por PYME:

```
GET /fiscal/contabilidad/libros
GET /fiscal/contabilidad/libros/{id}
```

Tres decisiones que hay que respetar:

- **La PYME se identifica por el token**, no por un `?tenant_id=`. CØRD Fiscal deriva el inquilino del JWT, así que el agente no puede pedir los libros de otra ni por descuido.
- **Solo libros confirmados**. Un libro sin confirmar es una interpretación que ningún humano validó.
- **Minimización en la traducción**. `datos_originales`, `categoria`, `problemas` y `proyecto` no se copian: el agente no los necesita para cotejar.

Y el aislamiento es comprobable, no declarativo:

```
python tools/verificar_frontera.py ../cord_rag_plataform/backend/app
(al 14-ago: 3 coincidencias en 1 062 líneas, las tres from datetime import ...).
```

6. Datos y persistencia — la brecha estructural

Capa	Local / CI	Cloud Run hoy	Producción documentada
Motor	SQLite	ninguno	PostgreSQL
Archivo	<code>bitacora.db</code>	no existe	servidor administrado
Migración	<code>Bitacora.migrar()</code> en cada petición	—	<code>migraciones/001_bitacora.sql</code>

Serialización	BEGIN IMMEDIATE	—	pg_advisory_xact_lock
Append-only	por disciplina del código	—	trigger que rechaza UPDATE/DELETE

Dos consecuencias que hay que tener presentes:

1. **Cloud Run no persiste nada.** El servicio desplegado no abre base de datos. Y si se desplegara el servicio de auditoría tal cual, SQLite escribiría en el sistema de archivos efímero del contenedor: cada instancia tendría su propia cadena y el autoescalado (`maxScale: 20`) las bifurcaría. **Desplegar la auditoría exige antes resolver la base de datos**, no es un paso opcional.
2. `migraciones/001_bitacora.sql` nunca se ha ejecutado contra nada. Es un documento de intención, correcto pero no probado.

7. Identidad, permisos y secretos

7.1 La única cuenta de servicio

`1031368580327-compute@developer.gserviceaccount.com` hace **tres papeles a la vez**: construye la imagen, corre el servicio y llama a Vertex.

Rol	Para qué
<code>roles/cloudbuild.builder</code>	Que Cloud Build lea el zip y publique la imagen
<code>roles/aiplatform.user</code>	Que el contenedor llame a Gemini

Es acotado —no tiene Editor— y eso es correcto. Lo que no es correcto es que build y runtime compartan identidad: quien comprometa el contenedor hereda permiso de publicar imágenes.

7.2 Exposición

```
gcloud run services get-iam-policy agente-cfdi-run
bindings:
- members: [allUsers]
  role: roles/run.invoker
```

`ingress: all + allUsers`. **Todo el mundo puede llamar `/api/chat` y `/api/cierre-diario`**. Es deliberado para que el jurado abra la URL, y está declarado en el README y en la evidencia — pero significa que hoy:

- cualquiera puede disparar el cierre diario;

- cualquiera puede consumir cuota de Vertex a costa del proyecto.

Cloud Scheduler tampoco manda token OIDC: su único distintivo es el header `User-Agent: Google-Cloud-Scheduler`, que cualquiera puede falsificar.

7.3 Secretos

Secreto	Dónde	Estado
<code>WALLET_PRIVATE_KEY</code>	Secret Manager	Existe, 0 versiones , no montado
<code>CORD_FISCAL_TOKEN</code>	previsto en Secret Manager	No creado — la fuente real no se usa
<code>GOOGLE_API_KEY</code>	<code>.env</code> local	Presente en texto plano en disco

Sobre el tercero: `.env` está en `.gitignore` (línea 151) y en `.dockerignore`, así que **no está en el repo ni en la imagen** — la contención funcionó. Pero la llave sigue en el disco de la máquina de desarrollo y corresponde a la vía de Gemini API que ya se abandonó. Lo sano es revocarla en la consola y borrar el archivo: una credencial que ya no se usa solo puede hacer daño.

8. Observabilidad

Pieza	Estado
Logs de request de Cloud Run	[OK] automáticos
Logs de intento de Scheduler	[OK] <code>AttemptStarted / AttemptFinished</code>
Retención	30 días — la corrida del 17-ago desaparece el 16-sep-2026
Logging estructurado de la app	[NO] ninguno. El código no emite una sola línea
Políticas de alerta	[NO] cero (verificado vía la API de Monitoring)
Trazas	[NO] API encendida, sin instrumentar

La consecuencia práctica: hoy, si el job empieza a devolver 500, se descubre mirando manualmente. `setup_sprint2.ps1` deja el filtro sugerido escrito en pantalla —`resource.type="cloud_scheduler_job" AND severity>=ERROR`— pero **la política nunca se creó**.

9. Integración continua

`.github/workflows/pruebas.yml`, en cada push y cada PR a `main`, con dos trabajos:

1. **pruebas** — matriz Python 3.11 y 3.13, `pytest tests/ -v`. Se prueban las dos versiones porque `requires-python = ">=3.11"` sin verificar sería una promesa vacía.
2. **demo** — levanta **uvicorn de verdad** y corre `tools/demo.py`. Existe porque `TestClient` no levanta un servidor real y ahí se escaparon dos fallos (ver ADR 0005). El script no solo comprueba que el flujo no truene: verifica que las desviaciones encontradas sean exactamente las plantadas, que el folio duplicado se rechace y que la prueba de inclusión no exponga folios ajenos.

El CI no despliega. No hay CD: el despliegue es manual, desde una máquina, con `deploy.ps1`. Cloud Build corre disparado por gcloud, no por un trigger de repositorio.

10. Inventario de brechas, por criticidad

#	Brecha	Consecuencia	Dónde
1	La auditoría no está desplegada	La URL pública no hace nada de lo que el proyecto promete	tarea 1.13
2	Sin persistencia en la nube	Desplegar la auditoría con SQLite bifurcaría la cadena entre instancias	§6
3	<code>/api/cierre-diario</code> es un stub	El job corre a diario y no cierra nada	<code>main.py:117</code>
4	Servicio público sin autenticación	Cualquiera dispara el cierre y consume cuota de Vertex	§7.2
5	Anclaje simulado	Sin él la cadena solo prueba consistencia interna, que es circular	ADR 0006
6	Cero alertas	Una falla silenciosa se descubre por casualidad	tarea 2.11

7	El agente no tiene herramientas	El LLM no puede consultar la bitácora que el proyecto construyó	<code>agent.py:45</code>
8	Dependencias divergentes	Al unificar los servicios, el arranque revienta al importar	\$4
9	Sin autenticación por financiador	IAM protege el perímetro, no distingue a un financiador de otro	<code>app.py:11</code>
10	Build y runtime comparten SA	Comprometer el contenedor da permiso de publicar imágenes	\$7.1

Las brechas 1, 2 y 3 son la misma historia contada tres veces, y están en orden: **no se puede integrar sin resolver antes la base de datos.**

11. Apéndice — verificar todo lo anterior

El SDK es portátil y hay dos cuentas en la máquina, así que hay que fijar la configuración explícitamente:

```
export PATH="$PATH:/d/CORD/tools/google-cloud-sdk/bin" && export
CLOUDSDK_CONFIG=/d/CORD/tools/gcloud-config-ricardo && gcloud config list
```

```
gcloud run services describe agente-cfdi-run --project project-d0428141-1b39-47af-9bc
--region us-central1 --format=yaml
```

```
gcloud run services get-iam-policy agente-cfdi-run --project
project-d0428141-1b39-47af-9bc --region us-central1
```

```
gcloud scheduler jobs describe job-cierre-diario --project
project-d0428141-1b39-47af-9bc --location us-central1
```

```
gcloud secrets versions list WALLET_PRIVATE_KEY --project project-d0428141-1b39-47af-9bc
```

```
gcloud projects get-iam-policy project-d0428141-1b39-47af-9bc
--flatten="bindings[].members" --format="table(bindings.role,bindings.members)"
--filter="bindings.members:1031368580327-compute"
```

Y el servicio, en vivo:

```
curl -s https://agente-cfdi-run-xsxcmt7edq-uc.a.run.app/
```

Variables de entorno

Variable	Valor en Cloud Run	Para qué
GOOGLE_GENAI_USE_VERTEXAI	1	Manda sobre todo lo demás
GOOGLE_CLOUD_PROJECT	project-d0428141-1b39-47af-9bc	Proyecto de facturación de Vertex
GOOGLE_CLOUD_REGION	global	Ubicación del modelo
GEMINI_MODEL	(sin fijar)	Default gemini-3.5-flash
AGENTE_CFDI_FUENTE	(sin fijar)	sintetica por omisión
AGENTE_CFDI_BITACORA	(sin fijar)	Ruta del SQLite. Default bitacora.db
AGENTE_CFDI_INQUILINO	(sin fijar)	Default DEM00000000XX0
CORD_FISCAL_URL / _TOKEN	(sin fijar)	Exigidas si la fuente es cord_fiscal
WALLET_PRIVATE_KEY	(sin fijar)	Reservada para el anclaje real

Equivocarse por omisión lleva siempre a la fuente sintética. Es deliberado: equivocarse hacia datos sintéticos es inofensivo; hacia datos reales, no.

Documentación relacionada

- [ADR 0001 — Serialización canónica](#)
- [ADR 0003 — Lectura de CFDI](#)
- [ADR 0004 — Bitácora encadenada](#)
- [ADR 0005 — Endpoints](#)
- [ADR 0006 — Anclaje y prueba](#)
- [Evidencia del job diario](#)
- [Contrato del expediente](#)
- [Frontera con CØRD Fiscal](#)